

Web Based Diabetes Prediction System Using Machine Learning

A Project Report

Submitted by:

AKASH PRADHAN (2141016398)

SUNNY SAURABH (2141019234)

RAHUL KUMAR (2141011138)

in partial fulfillment for the award of the degree

of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE & ENGINEERING



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
Faculty of Engineering and Technology, Institute of Technical Education and Research
SIKSHA 'O' ANUSANDHAN (DEEMED TO BE) UNIVERSITY
Bhubaneswar, Odisha, India
(June 2025)

CERTIFICATE

This is to certify that the project report titled “***Web Based Diabetes Prediction System Using Machine Learning***” being submitted by **Akash Pradhan, Sunny Saurabh, and Rahul Kumar** from section C to the Institute of Technical Education and Research, Siksha ‘O’ Anusandhan (Deemed to be) University, Bhubaneswar for the partial fulfillment for the degree of Bachelor of Technology in Computer Science and Information Technology is a record of original confident work carried out by them under my supervision and guidance. The project work, in my opinion, has reached the requisite standard fulfilling the requirements for the degree of Bachelor of Technology.

The results contained in this report have not been submitted in part or full to any other University or Institute for the award of any degree or diploma.

Mr. Maddu Naveen Kumar

Department of Computer Science and Information Technology

Faculty of Engineering and Technology;

Institute of Technical Education and Research;

Siksha ‘O’ Anusandhan (Deemed to be) University

ACKNOWLEDGEMENT

I am deeply grateful to ITER, Siksha 'O' Anusandhan University for providing the resources, infrastructure, and facilities that were essential for the successful completion of this project. The university's supportive environment and access to advanced technologies greatly contributed to the quality and outcome of my work.

I would like to extend my heartfelt thanks to our faculty in charge Dr. Dipankar Maity for his invaluable guidance, mentorship, and insightful suggestions throughout the development process.

I also wish to express my gratitude to my team members and peers for their collaboration, constructive feedback, and willingness to share ideas.

Thank you all for your support and encouragement!

Place: Bhubaneswar

Date: 28-03-2025

Signature of students

DECLARATION

We declare that this written submission represents our ideas in our own words and where other's ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/fact/source in our submission. We understand that any violation of the above will cause for disciplinary action by the University and can also evoke penal action from the sources which have not been properly cited or from whom proper permission has not been taken when needed.

Signature of Students with Registration Numbers :-

- | | |
|------------------|--------------|
| 1. Akash Pradhan | (2141016398) |
| 2. Sunny Saurabh | (2141019234) |
| 4. Rahul Kumar | (2141011138) |

Date: 19-09-2025

REPORT APPROVAL

This project report entitled “*Web Based Diabetes Prediction System Using Machine Learning*” by Akash Pradhan, Sunny Saurabh and Rahul Kumar is approved for the degree of Bachelor of Technology in Computer Science & Information Technology.

Examiners

ABSTRACT

Diabetes is a common and serious health condition influenced by factors such as age, obesity, physical inactivity, genetic predisposition, unhealthy lifestyle habits, high blood pressure, and poor diet. It raises the risk of severe complications, including heart disease, kidney failure, stroke, vision impairment, and nerve damage. Hospitals currently diagnose diabetes using various medical tests and provide treatment based on the results. Given the massive amount of data available in healthcare, Big Data Analytics plays a key role in detecting hidden patterns and trends that improve diagnostic accuracy and treatment efficiency. Analyzing large datasets helps uncover critical insights that enhance prediction models and lead to better healthcare decisions. This study introduces an advanced predictive model for diabetes classification, integrating additional external factors alongside traditional indicators like glucose levels, BMI, age, and insulin. By incorporating these extra variables, the model significantly improves classification and prediction accuracy over conventional methods. Furthermore, a diabetic pipeline model has been developed to enhance the precision of diabetes detection using advanced data analysis techniques. This data-driven approach enhances diagnostic reliability, improves diabetes classification, and ultimately leads to better treatment strategies. Implementing such models in healthcare enables more effective disease management, earlier detection, and improved patient outcomes, making a significant impact in combating diabetes.

	COURSE OUTCOME
CO1	Apply the acquired technical knowledge and skills to plan, analyze, design, and implement a software project or gather knowledge over the chosen field of research, and design or model solutions to proposed work.
CO2	Apply standard practices and strategies in software project development in designing and implementing a working, medium sized project as a team through simulation and/or experimental studies using modern tools and techniques.
CO3	Identify, analyze, formulate, and solve real-world problems creatively through sustainable critical investigation.
CO4	Demonstrate the ability to work as a team and communicate effectively in speech and writing through presentations and project reports.
CO5	Acquire the skills, diligence, and commitment to excellence needed to engage in lifelong learning.
CO6	Demonstrate an awareness and application of appropriate personal, societal, technical, and professional ethical standards in project development and management.

	Program Outcomes (PO) and Program Specific Outcomes (PSO)
POS	Description
PO1	Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
PO2	Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
PO3	Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
PO4	Conduct investigations of complex problems: Use research-based knowledge and research methods, including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
PO5	Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools, including prediction and modelling, to complex engineering activities with an understanding of the limitations.
PO6	The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal, and cultural issues, and the consequent responsibilities relevant to the professional engineering practice.
PO7	Environment and sustainability: Understand the impact of professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
PO8	Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
PO9	Individual and teamwork: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

PO10	Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
------	--

PO11	Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
PO12	Life-long learning: Recognize the need for and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.
PSO1	Modelling and Analysis: An ability to mathematically model and analyze the performance of Electrical Machines, Control systems, Instrumentation systems, Power systems, and Power Electronic systems.
PSO2	Design and Development: An ability to design the hardware and software requirements for the development of Electric drives, Automation and Embedded systems.

Table of Contents

	Title Page	
	Certificate of the Guide	i
	Declaration of the Student	ii
	Acknowledgement	Iii
	Report Approval	iv
	Abstract	v vi
	Course Outcome/Program Outcome	x i
	List of Figures	xi
	List of Tables (optional)	
	Chapter 1	1-3
	• Introduction • Background • Literature Review/Related product /process/algorithms/software, etc. • Problem Definition/Objective of work • Work Plan.	
	Chapter 2	4-10
2.	Design of Product/Process/Algorithm/Software, etc.	
	1. Alternative ideas 2. Design / Comparison Criteria 3. Evaluation for the selection of best idea 4. Detail design	
	Chapter 3	11-15
3.	Results and Discussion	
	Chapter 4	

4.	Conclusions and Future Scope.	16-17
5.	REFERENCES	18
6.	REFLECTION ON THE DESIGN PROCESS	19-20
7.	APPENDICES	21-47
8.	SIMILARITY REPORT	

List of Figure

Fig. No	Figure Name	Page No.
1	Predict	14
2	Diabetic	14
3	Non diabetic	14
4	Login	14
5	Signup	15

List of Tables

Fig. No.	Figure name	Page No.
1	idea evaluation	7
2	Isolation tree outlier	11
3	Local Outlier Detection and Removal	11
4	IQR Technique	12
5	Accuracy comparison	13

Chapter 1: Introduction

1.1

1. Background

Diabetes is a chronic metabolic disorder affecting millions worldwide, characterized by impaired glucose absorption due to insulin deficiency or resistance. According to the International Diabetes Federation, the number of diabetes cases is rapidly increasing due to factors such as urbanization, sedentary lifestyles, and poor dietary habits. In 2016, the WHO reported that 8.5% of adults had diabetes, leading to millions of deaths annually. The condition can cause severe complications, including cardiovascular diseases, kidney failure, and nerve damage, making early detection and management crucial.

Traditional diagnostic methods often face challenges like late detection and misdiagnosis. However, recent advancements in artificial intelligence (AI) and machine learning (ML) have enabled more efficient and accurate disease prediction. AI-powered tools can analyze vast amounts of patient data, helping healthcare professionals detect diabetes at an early stage and personalize treatment plans.

2. Literature Review

Aishwarya Mujumdar and Dr. Vaidehi et al.[1] work on various models such as Random Forest, Gradient Boost, SVM, Xg Boost, etc, and AdaBoost in 2019 and finds that AdaBoost gives the best performance

MD. Kamrul Hasan and MD. Ashraful Alam et al. [2] worked on various ML algorithms in 2020 and found that preprocessing and Robust Scaler work well in noisy data, and random forest gives the best result among all.

Muhammad Excell Febrian and Freansiskus Xaverius Fedinan et al.[3] Applied various models on the Pima dataset and concluded that the Naive Bayes Model is good for prediction in 2022

Jingyu Xue, Fancha Min, and Fengying Ma from Jinan, Shandong, China, et al.[4] found that there is no significant relation with age and diabetes, and SVM is the best-performing model for prediction

3. Problem Definition

Problem Definition:

The primary problem addressed in this project is the inefficiency and inaccuracy of traditional diabetes diagnosis methods. The lack of advanced predictive models in early detection and management poses several challenges:

- **Low accuracy and inefficiency** in traditional machine learning models lead to inconsistent diagnoses.
- **Feature selection and class imbalance issues** affect the reliability of diabetes prediction.
- **Manual assessments are time-consuming** and prone to human error.
- **Lack of real-time monitoring** results in delayed detection and treatment, increasing health risks.

To overcome these challenges, this project aims to develop an AI-powered diabetes prediction system using deep learning techniques, enabling highly accurate, automated, and real-time diagnosis for better healthcare management.

Objectives:

The objectives of this project include:

- Developing a Machine Learning Model to Detect Prediabetic Patients
- Improving model accuracy over pre-existing models by enhancing the dataset, applying optimized feature selection techniques, and experimenting with various machine learning algorithms to identify the most effective approach.
- Integrate the model with a web application to provide easy access for users, enabling seamless diabetes risk prediction and early detection through a user-friendly interface.

4. Work Plan

Phase 1: Data collection & Research

- Collecting the dataset. Check validation and go through features

Phase 2: Preprocessing Data

- Check null and inconsistent data and replace them

- Detect outliers and remove them

Phase 3: Feature Selection & Scaling

- Choose ML algo, check different tuning parameters
- Finding the best parameter through an algorithm
- Train various models and check accuracy
- Finalize ML model

Phase 4: Integrate with Frontend and Host the website

- Integrate with the Front end through Flask
- And Host over render

Chapter 2: Design of Product

1. Alternative ideas

In the process of developing a diabetes prediction system, the following alternative approaches were considered before finalizing the machine learning-based prediction model:

Idea 1: Manual Risk Assessment Using Standardized Questionnaires

Key Components:

- Clinically validated forms like the ADA Diabetes Risk Test
- Patient interviews or self-reported data collection
- Basic risk factor analysis (age, BMI, Blood Pressure, etc.)

Functionality:

Healthcare providers assess individuals using standardized questionnaires that evaluate known risk factors such as Pregnancy, Skin Thickness, Age , BMI, Diabetes Pedigree Function, and Blood Pressure. Based on a scoring threshold, patients are categorized into risk levels. While this approach is simple and cost-effective, it lacks personalization, cannot process large datasets efficiently, and may miss non-obvious risk patterns. It also requires significant manual effort and does not scale well for large populations.

Idea 2: Rule-Based Expert System for Diabetes Diagnosis

Key Components:

- A set of predefined medical rules
- Symptom and history input module
- Logical inference engine

Functionality:

This system uses a fixed set of medical rules derived from clinical guidelines. Users input data such as symptoms, medical history, and test results, which the system evaluates against its rule base to determine the likelihood of diabetes. While easier to validate medically and explain to users, rule-based systems are rigid and struggle to adapt to diverse or evolving patient data. They also fail to uncover complex correlations hidden in large datasets.

Idea 3: Machine Learning-Based Diabetes Prediction Model (Final Choice)**Key Components:**

- Historical health data (e.g., from the PIMA Indian Diabetes dataset)
- Data preprocessing and feature selection pipeline
- ML algorithms (e.g., Random Forest, Logistic Regression, XG Boost)
- Prediction dashboard or API integration

Functionality:

Patient data such as glucose levels, blood pressure, BMI, insulin levels, and more are fed into a trained machine learning model. The system predicts the likelihood of diabetes based on learned patterns and outputs a risk score. This approach enables high accuracy, personalization, and scalability. It also allows real-time prediction, integrates well with health platforms, and can be continuously improved with more data. The model can highlight feature importance, enabling interpretability and data-driven preventive care.

2. Comparison Criteria

To evaluate and compare the proposed ideas, the following criteria will be used:

- **Automation level:** Measures the extent to which the system minimizes manual input, analysis, and decision-making by leveraging automated tools or algorithms.
- **Accuracy and reliability:** Assesses how precisely the system can predict diabetes risk, including its performance across different patient demographics and data conditions.
- **Data adaptability:** Evaluates the system's ability to handle diverse input data types (e.g., missing values, outliers, varying formats) and adjust to updated or real-time data.

- **Interpretability:** Considers how clearly the system can explain its predictions or decisions to medical professionals and patients, which is crucial for trust and clinical validation.
- **Scalability:** Determines how effectively the system can scale to handle larger populations or datasets without performance degradation.
- **Ease of implementation:** Considers the complexity of setting up the system, including required technical expertise, infrastructure, and integration into clinical workflows.
- **Cost effectiveness:** Takes into account both initial development or deployment costs and long-term maintenance or data update expenses.
- **Real-time prediction capability:** Assesses whether the system can process patient data and return predictions or risk scores instantly or with minimal delay.
- **Integration with electronic health records (EHRs):** Checks how seamlessly the system can be integrated with existing digital health infrastructure to streamline operations.

3. Evaluation for the selection of the best idea

The three proposed ideas were evaluated using the following fixed criteria to determine their feasibility and effectiveness:

Criteria	Manual Risk Assessment	Rule-Based Expert System	ML-Based Prediction Model (Selected)
Automation level	Low – requires manual data entry and scoring	Medium – automatic logic, manual inputs required	Very High – fully automated from input to prediction
Accuracy and reliability	Low – depends on subjective self- reports	Medium – limited by predefined rules	High – learns complex patterns from real data
Data adaptability	Low – not designed for large or varied datasets	Low–rigid structure limits flexibility	High – handles missing values, updates with new data

Interpretability	Highly simple and transparent scoring	High – based on logical medical rules	Medium – explainable via feature importance methods
Scalability	Low – not practical for mass screening	Medium – deployable but not dynamic	Highly scalable across populations and platforms
Ease of implementation	High – easy to deploy, no tech required	Medium – needs software logic setup	Medium – requires model training and a data pipeline
Cost effectiveness	High – minimal cost, manual effort only	Medium – development time and logic upkeep	High initial investment, low cost at scale
Real-time prediction capability	Not Applicable	Low-level checking is not instant	High – provides instant predictions on input
Integration with EHRs	Low – not designed for digital systems	Medium – can be built into health platforms	High – easily integrates with digital records

Figure 1 Idea Evaluation

Conclusion:

Based on the evaluation:

- **Manual Risk Assessment:** It is easy to implement and highly interpretable, but lacks automation, scalability, and accuracy, making it unsuitable for large-scale or data-driven environments.
- **Rule-Based Expert System:** It offers moderate automation and clear logic but struggles with adaptability and fails to capture complex patterns in diverse patient data.
- **Machine Learning-Based Prediction Model:** It excels in automation, accuracy, data adaptability, scalability, and integration with digital health systems. It provides real-time risk prediction and supports continuous improvement as more data becomes available.

Therefore, the **Machine Learning-Based Prediction Model** is selected as the most effective, scalable, and future-ready solution for developing a robust diabetes prediction system.

4. Detailed design

Frontend Design:

- **Design of React Components**
 - Signup
 - Login
 - Home
 - Diabetic
 - Nondiabetic
 - Navbar

- **Create Routes**
 - /
 - /login
 - /signup
 - /home
 - /diabetic
 - /nondiabetic

Backend Design:

- **Database Connection:**

Connect with mongodb Atlas for data storage for login and signup.
- **Routes Create:**
 - /login
 - /signup
 - /predict

Data Preprocessing and Model Training:

- **Data Set Collection:**
 - The data set that we used is named as PIMA India. It is given by the National Institute of Diabetes and Digestive and Kidney Diseases.
 - It has 768 rows and 9 columns.
 - The features are

- Pregnancies
 - Glucose-Plasma glucose concentration a 2 hours in an oral glucose tolerance test
 - Blood Pressure
 - Skin Thickness -Triceps skin fold thickness
 - Insulin- 2-Hour serum insulin
 - BMI-weight in kg/ (height in m)^2
 - Diabetes Pedigree Function- It shows the diabetic history of your family
 - Age
- **Data Preprocessing:**
 - **Find out Missing values and replace:**
 - In this data, there were no missing values, but some zeroes are considered invalid in features (Glucose, Age, Insulin, BMI)
 - We replaced those data with their mean, median, mode, and class mean.
 - **Find out outliers and remove them:**
 - We check for outliers with Local Outlier Factor detection and removal, Isolation Tree Outlier detection and removal, and the IQR Method
 - Then choose the IQR method
 - **Add Additional Features**
 - We add additional features based on preexisting features as NewInsulinScore. If insulin is more than 166, then it would be abnormal else normal.
 - Then, based on BMI, we add NewBMI
 - Then, based on glucose, add NewGlucose
 - **Perform one-hot Encoding and Scaling**
 - Then convert the categorical data into numerical data by using one-hot encoding.
 - And use standard scalar and robust scaler for scaling the noncategorical data
- **Model Selection and Training**
 - Random Forest
 - Balanced Random Forest
 - XG Boost
 - Gradient Boosting

- Ada Boost
 - Stacking
- **Prediction and Output:**
 - Then we choose a input array
 - Then add new columns as we used in the training data
 - Then use one-hot encoding and the same scaler to scale the data
 - Then predict by the output
- **Interface and Integration:**
 - **Web-based dashboard** for patients or healthcare professionals to input medical data (e.g., glucose, insulin, BMI) and view real-time diabetes risk predictions.
 - **REST API endpoints** developed using Flask, enabling seamless integration with hospital management systems, telehealth platforms, or mobile health apps.

Chapter 3: Results and Discussion

In the data set, there were a lot of zero values. So we replaced it with their mean values regarding their classes. As it gives me the best result

- **Outlier detection and removal**

Table 2 Isolation tree outlier

Model	Accuracy
Random Forest	0.86
XG Boost	0.86
Ada Boost	0.88
Gradient Boosting	0.89
Stacking	0.87
Balanced Random Forest	0.84

Table 3 Local Outlier Detection and Removal

Model	Accuracy
Random Forest	0.89
XG Boost	0.88
Ada Boost	0.88
Gradient Boosting	0.89
Stacking	0.89
Balanced Random Forest	0.90

Table 4 IQR Technique

Model	Accuracy
Random Forest	0.92
XG Boost	0.92
Ada Boost	0.95
Gradient Boosting	0.94
Stacking	0.91
Balanced Random Forest	0.92

So, from this observation, we found that the IQR technique is best for our dataset.

Different Models

- **Random Forest:**
 1. It is a bagging technique which have decision trees as its classifier
 2. `rfclf2=RandomForestClassifier(max_depth=12,max_features=0.3,max_samples=1.0,n_estimators=100,bootstrap=True)`
 3. These are the best parameters that I found from gridsearch cv
- **Ada Boost:**
 4. AdaBoost is a boosting technique that uses decision stumps as weak learners to build a strong classifier by focusing more on misclassified instances in each iteration.
 5. `AdaBoostClassifier(n_estimators=1500,learning_rate=0.1)`
 6. This is the best parameters that I get from randomsearch cv
- **Gradient Boosting:**
 7. Gradient Boosting is a powerful machine learning technique that builds an ensemble of decision trees sequentially, where each new tree is trained to correct the errors made by the previous trees. It does this by minimizing a loss function using gradient descent, hence the name.
 8. `gb_classifier=GradientBoostingClassifier( subsample=0.9,n_estimators=100,min_samples_split=5,min_samples_leaf=2,max_depth=3,learning_rate=0.15,random_state=42)`

- **XG boost:**
 3. XG Boost is a faster, more efficient version of gradient boosting that builds trees sequentially to correct prediction errors and improve accuracy.
 4. XGboost_classifier =

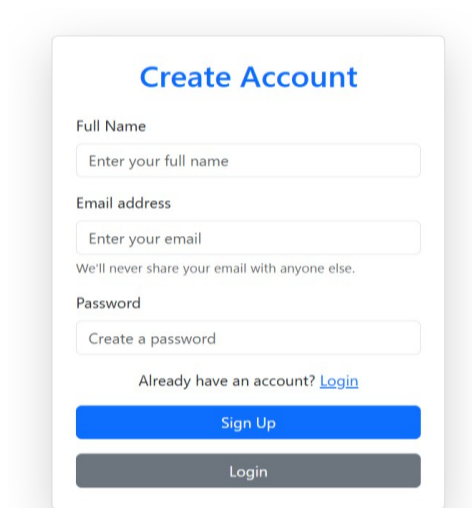
```
xgb.XGBClassifier( subsample=0.7, reg_lambda=0.1,
reg_alpha=0.001,
n_estimators=100,max_depth=6,learning_rate=0.01,
gamma=0,colsample_bytree=0.8, eval_metric='logloss' )
```
 5. These are the best parameters that I found
- **Balanced Random Forest:**
 9. Balanced Random Forest is an ensemble learning method designed to handle imbalanced datasets by combining the power of random forests with resampling techniques. It builds each decision tree on a balanced bootstrap sample, where the minority class is oversampled or the majority class is undersampled, helping the model learn both classes more effectively.

Choosing the Best Model

Table 5 Accuracy comparison

Algo	Accuracy	Class	Precision	Recall	F1-Score
Random Forest	89%	0	0.93	0.94	0.93
		1	0.87	0.85	0.86
XG Boost	91%	0	0.90	0.96	0.93
		1	0.91	0.80	0.85
Ada Boost	95%	0	0.96	0.96	0.96
		1	0.93	0.93	0.93
Gradient Boost	94%	0	0.96	0.95	0.96
		1	0.90	0.93	0.91
Stacking	92%	0	0.93	0.96	0.94
		1	0.92	0.85	0.88
Balanced Random Forest	91%	0	0.96	0.90	0.93
		1	0.82	0.93	0.87

From this observation, we conclude that the **AdaBoost Classifier** is the best model.



Create Account

Full Name

Email address

We'll never share your email with anyone else.

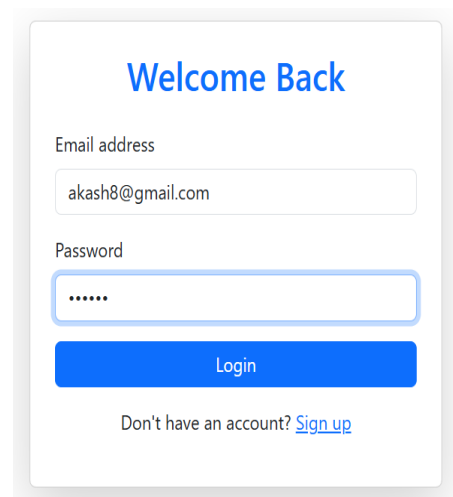
Password

Already have an account? [Login](#)

[Sign Up](#)

[Login](#)

Figure 1 Create Account



Welcome Back

Email address

Password

[Login](#)

Don't have an account? [Sign up](#)

Figure 2 Login



Diabetes Prediction
 Enter your details to check your risk

Number of Pregnancies

Glucose Level

Blood Pressure

Skin Thickness

Insulin Level

Body Mass Index (BMI)

Diabetes Pedigree Function

Age

[Predict](#)

Figure 3 Predict

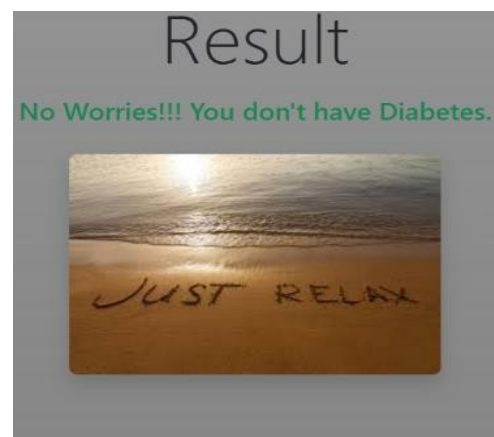


Figure 4 Non-Diabetic

Result

Chances of having Diabetes is more, please consult a Doctor.



Figure 5 Diabetic

Chapter 4: Conclusions and Future Scope.

Conclusions:

In the health-care business, early detection and accuracy have a great impact on the patient's future. We developed a technology that can accurately predict diabetes, specifically Type 1 and Type 2. In our search, first we clean the data with the IQR method, then add some new features, then use one-hot encoding and raw burst scaler. This data is trained on 6 different models to check different aspects. Among them, AdaBoost does well with some hyperparameter tuning. We integrate this model into our website, where users can log in and give the required details, and can check the result within a few seconds. This may help the medical professionals to reduce their workload and support timely. And ease them in treating the patients.

Future Scope for Improvement:

- Add a chatbot for patient education using retrieval-augmented generation (RAG).
- Enable a reminder system for exercise and diet based on individual risk
- Work on the big dataset to improve accuracy
- We can apply deep learning models to improve accuracy

REFERENCES

1. Majumdar, A., & V, V. (2019). Diabetes Prediction using Machine Learning Algorithms. International Conference on Recent Trends in Advanced Computing (ICRTAC 2019). | A. Majumdar and V. Vaidehi, "Diabetes Prediction using Machine Learning Algorithms," 2019 International Conference on Recent Trends in Advanced Computing (ICRTAC 2019), 2019. | Majumdar, Aishwarya, and Vaidehi V. "Diabetes Prediction Using Machine Learning Algorithms." International Conference on Recent Trends in Advanced Computing (ICRTAC 2019), 2019.
2. Hasan, M. K., Alam, M. A., Das, D., Hossain, E., et al. (2020). Diabetes prediction using ensembling of different machine learning classifiers. IEEE Access. Retrieved from ieeexplore.ieee.org.
3. Sonar, P., & JayaMalini, K. (2019). Diabetes prediction using different machine learning approaches. 2019 3rd International Conference on Computing and Communications Technologies (ICCCT). IEEE. Retrieved from ieeexplore.ieee.org.
4. J. Xue, F. Min, and F. Ma, "Research on diabetes prediction method based on machine learning," Journal of Physics
5. R. Miotto, F. Wang, S. Wang, X. Jiang, T. Dudley, Deep learning for healthcare: review, opportunities and challenges. *Brief. Bioinform.* 19(6), 1236–1246 (2017)
6. D. Fagella, 7 Applications of Machine Learning in Pharma and Medicine, 2017, [online] Available: <https://goo.gl/IS1R5k>
7. G.D. Magoulas and A. Prentza, "Machine learning in medical applications" in *Machine Learning and its applications*, Springer Berlin Heidelberg, pp. 300-307, 2001. 25 ess, New
8. Fatima, Meherwar, and Maruf Pasha. "Survey of machine learning algorithms for disease diagnosis." *Journal of Intelligent Learning Systems and Applications* 9.01 (2017).

REFLECTION ON THE DESIGN PROCESS

Reflection on our project web-based diabetes prediction using ML include various steps the detailed reflection is given below

1. Problem Understanding and Objective

We want to develop a web based app that detect the diabetic using patients given data. For that we have to train a ml algo on data set then integrate with frontend and backend.

2. Design Process

a. User registration

The user should register first with the web and login into home page. He/she have to give necessary values of input field and press prediction.

b. Prediction

Then the data collected and send to the flask server where it hit on predict route where data get modified by add additional feature, one-hot encoding and scaling then go for the model

c. Model Development

We trained the model on PIMA India dataset. First we go through he data set how data is distributed how the feature affect the data.

d. Data Preprocessing

We check null values and replace with their class median. Then check for outliers using the IQR method then remove it.

e. Model Training

We go for various algo and try wth hyperparameter tuning that would give us best result. Among all 6 model we get AdaBoost give us best result.

f. Model Training

After that we integrate the model with flask server using pickle file.

g. Output

As per the prediction the user would redirect to the diabetic or nondiabetic.

3. Overall Reflection

Strengths

As we use AdaBoost classifier as our model it would give us accuracy 95%. And the output would give us within seconds

We use website to integrate with user it would reduce waiting timing. And friendly experience.

Weakness

The taken dataset is not so large it would be better for having bigger dataset.

The model has not been tested with real clinical data or evaluated in collaboration with healthcare professionals, which limits its clinical readiness.

There's no mention of testing on an external dataset, which is important for proving the model's ability to generalize beyond the training data.

APPENDICES

CODE ATTACHMENTS

FRONTEND:

Main.jsx:

```
import { StrictMode } from 'react'

import { createRoot } from 'react-dom/client'

import App from './App.jsx'

createRoot(document.getElementById('root')).render(

  <StrictMode>

    <App />

  </StrictMode>,

)
```

App.jsx:

```
import { useState } from "react";

import Signup from "./LoginSignup/Signup";

import Login from "./LoginSignup/Login"

import Home from "./LoginSignup/Home"

import "../src/index.css";

import "../App.css";

import { BrowserRouter, Routes, Route ,Link} from "react-router-dom";

function App() {

  const [count, setCount] = useState(0);

  return (

    <div>
```

```

    <BrowserRouter>

    <Routes>

    <Route path="/" element={<Signup/>} />

    <Route path="/Login" element={<Login/>} />

    <Route path="/Signup" element={<Signup/>} />

    <Route path="/Home" element={<Home/>} />

    </Routes>

  </BrowserRouter>

</>

);
}

```

export default App;

Home.jsx:

```

import React from 'react'

import NavBar from './NavBar'

function Home() {

  return (

    <div >

      <NavBar/>

      <div className='home'>

        <h2>WellCome !</h2>

        <button className='btn'>LogOut</button>

      </div>

    </div>

  )

```

```
}
```

```
export default Home
```

Login.jsx:

```
import React from 'react'
```

```
import NavBar from './NavBar'
```

```
import main from '../images/main.jpg'
```

```
import {Link} from "react-router-dom";
```

```
function Login() {
```

```
  return (
```

```
    <div>
```

```
      <NavBar/>
```

```
      <div className='main-page'>
```

```
        <form action="">
```

```
          <div className='heading'>
```

```
            Login
```

```
          </div>
```

```
          <div>
```

```
            <input type="email" name='Email' placeholder='Enter your Email' />
```

```
            <input type="password" name='Password' placeholder='Enter your
```

```
password' />
```

```
            <p>Create account <Link to='/Signup'>Sgnup</Link></p>
```

```
          </div>
```

```
          <button className='btn'>Login</button>
```

```
        </form>
```

```
        <img src={main} alt="img" />
```

```
        </div>

    </div>

)
}
```

```
export default Login
```

Navbar.jsx:

```
import React from "react";

function NavBar()
{ return (
    <div>

        <nav>

            <ul>

                <li>

                    <a className="head">Explore!</a>

                    <a>Signup</a>

                    <a>Login</a>

                </li>

            </ul>

        </nav>

    </div>

);
}
```

```
export default NavBar;
```

Signup.jsx:

```
import React from 'react'

import main from '../images/main.jpg'

import NavBar from './NavBar'

import {Link} from "react-router-dom"

function Signup() {

  return (

    <div>

      <NavBar/>

      <div className='main-page'>

        <form action="">

          <div className='heading'>

            SIGNUP

          </div>

          <div>

            <input type="text" name='Name' placeholder='Enter your name' />

            <input type="email" name='Email' placeholder='Enter your Email' />

            <input type="password" name='Password' placeholder='Enter your password' />

            <p>Already have an account <Link to='/Login'>Login</Link></p>

          </div>

          <button className='btn'>SignUP</button>

        </form>

        <img src={main} alt="img" />

      </div>

    </div>

  )

}
```

```
}
```

```
export default Signup
```

App.css

```
* {
```

```
padding: 0;
```

```
margin: 0;
```

```
box-sizing: border-box;
```

```
}
```

```
.btn {
```

```
padding: 12px 24px;
```

```
background-color: #4f46e5; /* Indigo */
```

```
color: white;
```

```
border: none;
```

```
border-radius: 8px;
```

```
font-size: 16px;
```

```
font-weight: 600;
```

```
cursor: pointer;
```

```
transition: all 0.3s ease;
```

```
box-shadow: 0 4px 10px rgba(0, 0, 0, 0.1);
```

```
}
```

```
.btn:hover {
```

```
background-color: #4338ca; /* Darker Indigo */
```

```
transform: translateY(-2px);
```

```
box-shadow: 0 6px 14px rgba(0, 0, 0, 0.15);
```

```
}
```

```
.btn:active {  
  transform: translateY(0);  
  box-shadow: 0 3px 8px rgba(0, 0, 0, 0.12);  
}
```

```
.home{ displa  
  y: flex;  
  flex-wrap: wrap;  
  align-items: center;  
  justify-content: space-around;  
  margin: 30px;  
}
```

Index.css:

```
form{  
  width: 330px;  
  text-align: center;  
}
```

```
form  
  input{ width:  
    95%; padding:  
    5px;  
    border: 1px solid rgb(119, 119, 119);  
    font-size: 17px;  
    color: rgb(83, 83, 83);  
    margin-top: 21px;
```

```

}
form
  button{ width:
    135px; height:
    33px;
    background-color: rgb(2, 173, 122);
    color: white;
    font-size: 18px;
    border: none;
    cursor: pointer;
    margin-top: 22px;
  }

```

```

form .heading{ fo
  nt-size: 30px;
  font-weight: 600;
  margin-bottom: 20px;
}

```

```

.main-
  page{ display
    : flex;
    justify-content: center;
    align-items: center;
    gap: 50px;
    margin-top: 29px;
    flex-wrap: wrap;
  }

```

```

img{
  width: 500px;

```



```

}

nav{

    display: flex;

    justify-content: left;

    background-color: rgb(7, 7, 7);

    padding: 11px;

}

nav ul li

    a{ color:

        white;

        margin-inline: 11px;

    }

.head{

    font-size: 20px;

}

```

BACKEND

App.py

```

from flask import Flask, request, jsonify

from flask_cors import CORS

from pymongo import MongoClient

from dotenv import load_dotenv

import numpy as np

import pandas as pd

import pickle

import os

load_dotenv()

```

```

app = Flask(__name__)

CORS(app)

model = pickle.load(open("adaboost_model.pkl", 'rb'))

scaler = pickle.load(open("scaler.pkl", 'rb'))

# Connect to MongoDB Atlas

client = MongoClient(os.getenv("MONGO_URI"))

db = client.get_database("Diebeties") # Make sure this matches your DB name

people_collection = db.peoples


@app.route('/register', methods=['POST'])
def register():

    data = request.json

    name = data.get("name")

    email = data.get("email")

    password = data.get("password")

    print(name)

    print(email)

    print(password)

    print("Request JSON:", request.json)

    print("Parsed email:", email)


if people_collection.find_one({"email": email}):

    print(f"Checking for email: {email}")

    user = people_collection.find_one({"email": email})

    print("MongoDB returned:", user)

```

```

        return jsonify("exist")
    else:
        people_collection.insert_one({ "name": name,
                                        "email": email,
                                        "password": password
                                    })
    return jsonify("Success")

```

```
@app.route('/login', methods=['POST'])
```

```
def login():
```

```

    data = request.json
    email = data.get("email")
    password = data.get("password")
    user = people_collection.find_one({"email": email})
    if user:
        if user["password"] == password:
            return jsonify("Success")
        else:
            return jsonify("Incorrect password")
    else:
        return jsonify("Please create an account first")

```

```
@app.route("/predict", methods=["POST"])
```

```
def predict():
```

```

    data = request.get_json()
    print("Received JSON:", data) # Debug log

```

```

input_data = data.get("input_data")

input_data = input_data =
[ int(data["Pregnancies"]),
  float(data["Glucose"]),
  float(data["BloodPressure"]),
  float(data["SkinThickness"]),
  float(data["Insulin"]),
  float(data["BMI"]),
  float(data["DiabetesPedigreeFunction"]),
  int(data["Age"])
]

print(input_data)

ans1=not input_data

print(ans1)

# if not input_data or len(input_data) != 8:

#   print("not 8")

#   return jsonify({"error": "Invalid input data"}), 400


# Define column names

columns = ['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', 'Insulin',

           'BMI', 'DiabetesPedigreeFunction', 'Age']

input_df = pd.DataFrame([input_data], columns=columns)



```

Page **32** of **47**

```

# Required dummy columns

```

```

required_dummies = [
    'NewBMI_Obesity 1', 'NewBMI_Obesity 2', 'NewBMI_Obesity 3',
    'NewBMI_Overweight',
    'NewBMI_Underweight', 'NewInsulinScore_Normal', 'NewGlucose_Low',
    'NewGlucose_Normal', 'NewGlucose_Overweight', 'NewGlucose_Secret',
    'NewBMI_Normal', 'NewInsulinScore_Abnormal', 'NewGlucose_High'
]

```

```

for col in required_dummies:

```

```

    input_df[col] = False

```

```

# BMI Category

```

```

bmi = input_df.at[0, "BMI"]

```

```

if bmi < 18.5:

```

```

    input_df.at[0, "NewBMI_Underweight"] = True

```

```

elif 18.5 < bmi <= 24.9:

```

```

    input_df.at[0, "NewBMI_Normal"] = True

```

```

elif 24.9 < bmi <= 29.9:

```

```

    input_df.at[0, "NewBMI_Overweight"] = True

```

```

elif 29.9 < bmi <= 34.9:

```

```

    input_df.at[0, "NewBMI_Obesity 1"] = True

```

```

elif 34.9 < bmi <= 39.9:

```

```

    input_df.at[0, "NewBMI_Obesity 2"] = True

```

```

elif bmi > 39.9:

```

```

    input_df.at[0, "NewBMI_Obesity 3"] = True

```

```

# Insulin Score

```

```

insulin = input_df.at[0, "Insulin"]

if 16 <= insulin <= 166:

    input_df.at[0, "NewInsulinScore_Normal"] = True
else:

    input_df.at[0, "NewInsulinScore_Abnormal"] = True


# Glucose Category
glucose = input_df.at[0, "Glucose"]

if glucose <= 70:

    input_df.at[0, "NewGlucose_Low"] = True
elif 70 < glucose <= 99:

    input_df.at[0, "NewGlucose_Normal"] = True
elif 99 < glucose <= 126:

    input_df.at[0, "NewGlucose_Overweight"] = True
elif 126 < glucose <= 200:

    input_df.at[0, "NewGlucose_Secret"] = True
else:

    input_df.at[0, "NewGlucose_High"] = True


# Scale numerical data
numerical_df = input_df[columns]

scaled_numerical = scaler.transform(numerical_df)

scaled_df = pd.DataFrame(scaled_numerical, columns=columns)


# Get dummy columns
categorical_df = input_df[required_dummies]

```

```

# Merge for final input

final_input = pd.concat([scaled_df, categorical_df], axis=1)


# Make prediction

prediction = model.predict(final_input)[0]

result = "Positive" if prediction == 1 else "Negative"

print(result)

return jsonify({"prediction": result})


if __name__ == '__main__':

    app.run(port=3001, debug=True)

```

Model Training

```

import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.model_selection import train_test_split

from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

from sklearn.ensemble import RandomForestClassifier

from mlxtend.plotting import plot_decision_regions

```

```

from sklearn.datasets import make_circles

# Read Data set

data=pd.read_csv('/content/diabetes_dataset.csv')

# Histplot for data distribution

plt.figure(figsize=(12,12))

for i, col in enumerate(['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', 'Insulin',
'BMI', 'DiabetesPedigreeFunction', 'Age']):

    plt.subplot(3, 4, i+1)

    sns.histplot(x=col,data=data,kde=True)

plt.show()

# Nullvalues replace with zeros

def median_target(var):

    temp = data[data[var].notnull() ]#take the value where var value in not null

    temp = temp[[var, 'Outcome']].groupby(['Outcome'])[var].median().reset_index()

    #then keep only var and output column then groupby basis on outcome then find its median
    and return it

    return temp

# The values to be given for incomplete observations are given the median value of people who
are not sick and the median values of people who are sick.

columns = data.columns

columns = columns.drop("Outcome")

# Zedo values are replaced with the median value of the target variable.

for i in columns:

    median_target(i)

    data.loc[(data['Outcome'] == 0 ) & (data[i].isnull()), i] = median_target(i)[i][0]

    data.loc[(data['Outcome'] == 1 ) & (data[i].isnull()), i] = median_target(i)[i][1]

plt.figure(figsize=(12,12))

```

```
for i, col in enumerate(['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', 'Insulin',
'BMI', 'DiabetesPedigreeFunction', 'Age']):
```

```
    plt.subplot(4, 2, i+1)
```

```
    sns.histplot(x=col, data=data, kde=True)
```

```
plt.show()
```

```
columns = ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI',
'DiabetesPedigreeFunction']
```

```
# Outlier removal using IQR method
```

```
for col in columns:
```

```
    Q1 = data[col].quantile(0.25)#25 persentile
```

```
    Q3 = data[col].quantile(0.75)#75 persentile
```

```
    IQR = Q3 - Q1
```

```
    lower_bound = Q1 - 1.5 * IQR
```

```
    upper_bound = Q3 + 1.5 * IQR
```

```
    # Keep only the rows within the IQR range
```

```
    data = data[(data[col] >= lower_bound) & (data[col] <= upper_bound)]
```

```
print("Shape after outlier removal:", data.shape)
```

```
plt.figure(figsize=(12,12))
```

```
for i, col in enumerate(['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', 'Insulin',
'BMI', 'DiabetesPedigreeFunction', 'Age']):
```

```
    plt.subplot(4, 2, i+1)
```

```
    sns.histplot(x=col, data=data, kde=True)
```

```
plt.show()
```

```
# Add additional features
```

```
# According to BMI, some ranges were determined and categorical variables were assigned.
```

```
NewBMI = pd.Series(["Underweight", "Normal", "Overweight", "Obesity 1", "Obesity 2",  
"Obesity 3"], dtype = "category")
```

```
data["NewBMI"] = NewBMI
```

```
data.loc[data["BMI"] < 18.5, "NewBMI"] = NewBMI[0]
```

```
data.loc[(data["BMI"] > 18.5) & (data["BMI"] <= 24.9), "NewBMI"] = NewBMI[1]
```

```
data.loc[(data["BMI"] > 24.9) & (data["BMI"] <= 29.9), "NewBMI"] = NewBMI[2]
```

```
data.loc[(data["BMI"] > 29.9) & (data["BMI"] <= 34.9), "NewBMI"] = NewBMI[3]
```

```
data.loc[(data["BMI"] > 34.9) & (data["BMI"] <= 39.9), "NewBMI"] = NewBMI[4]
```

```
data.loc[data["BMI"] > 39.9, "NewBMI"] = NewBMI[5]
```

```
def set_insulin(row):
```

```
    if row["Insulin"] >= 16 and row["Insulin"] <= 166:
```

```
        return "Normal"
```

```
    else:
```

```
        return "Abnormal"
```

```
# Some intervals were determined according to the glucose variable and these were assigned  
categorical variables.
```

```
NewGlucose = pd.Series(["Low", "Normal", "Overweight", "Secret", "High"], dtype =  
"category")
```

```
data["NewGlucose"] = NewGlucose
```

```
data.loc[data["Glucose"] <= 70, "NewGlucose"] = NewGlucose[0]
```

```
data.loc[(data["Glucose"] > 70) & (data["Glucose"] <= 99), "NewGlucose"] = NewGlucose[1]
```

```
data.loc[(data["Glucose"] > 99) & (data["Glucose"] <= 126), "NewGlucose"] =  
NewGlucose[2]
```

```
data.loc[data["Glucose"] > 126, "NewGlucose"] = NewGlucose[3]
```

Here, by making One Hot Encoding transformation, categorical variables were converted into numerical values. It is also protected from the Dummy variable trap.

```
data = pd.get_dummies(data, columns=["NewBMI", "NewInsulinScore", "NewGlucose"])
```

```
categorical_data = data[['NewBMI_Obesity 1', 'NewBMI_Obesity 2', 'NewBMI_Obesity 3',  
'NewBMI_Overweight', 'NewBMI_Underweight',
```

```
                        'NewInsulinScore_Normal', 'NewGlucose_Low', 'NewGlucose_Normal',  
'NewGlucose_Overweight',  
'NewGlucose_Secret', 'NewBMI_Normal', 'NewInsulinScore_Abnormal', 'NewGlucose_High']]
```

```
Y = data["Outcome"]
```

```
x = data.drop(["Outcome", 'NewBMI_Obesity 1', 'NewBMI_Obesity 2', 'NewBMI_Obesity 3',  
'NewBMI_Overweight', 'NewBMI_Underweight',
```

```
                'NewInsulinScore_Normal', 'NewGlucose_Low', 'NewGlucose_Normal',  
'NewGlucose_Overweight',  
'NewGlucose_Secret', 'NewBMI_Normal', 'NewInsulinScore_Abnormal', 'NewGlucose_High'],  
axis = 1)
```

```
cols = x.columns
```

```
index = x.index
```

The variables in the data set are an effective factor in increasing the performance of the models by standardization.

There are multiple standardization methods. These are methods such as " Normalize", " MinMax", " Robust" and "Scale".

```
from sklearn.preprocessing import RobustScaler
```

```
transformer = RobustScaler().fit(x)
```

```
x = transformer.transform(x)
```

```
x = pd.DataFrame(x, columns = cols, index = index)
```

```

X= pd.concat([x,categorical_data], axis = 1)

# Train test split

x_train,x_test,y_train,y_test=train_test_split(X,Y,test_size=0.2,random_state=42)

# Train Random Forest Classifier

rfclf2=RandomForestClassifier(max_depth=12,max_features=0.3,max_samples=1.0,n_estimators=100,bootstrap=True)

rfclf2.fit(x_train,y_train)

x_train_prediction=rfclf2.predict(x_train)

x_train_accuracy=accuracy_score(x_train_prediction,y_train)

print("Accuracy Score for training data is ",x_train_accuracy)

x_test_prediction=rfclf2.predict(x_test)

x_test_accuracy=accuracy_score(x_test_prediction,y_test)

print("Accuracy Score for testing data is ",x_test_accuracy)


cm = confusion_matrix(y_test, y_pred, labels=[1, 0])

TP, FN, FP, TN = cm.ravel()

print("Modified Confusion Matrix (TP in [0][0]):")

print(f'[[TP: {TP}  FN: {FN}]]')

print(f'[[FP: {FP}  TN: {TN}]]')


print("\nClassification Report:\n", classification_report(y_test, y_pred))

```

```

import xgboost as xgb

from sklearn.model_selection import RandomizedSearchCV

from sklearn.model_selection import GridSearchCV

model = xgb.XGBClassifier(use_label_encoder=False, eval_metric='logloss')

param_dist = {

    'max_depth': [3, 4, 5, 6, 7],

    'learning_rate': [0.001, 0.01, 0.05, 0.1,0.15,0.2],

    'n_estimators': [100, 200, 300, 500,600 ,700, 900, 1200, 1500, 1800],

    'subsample': [0.6, 0.7, 0.8, 0.9, 1],

    'colsample_bytree': [0.6, 0.7, 0.8, 0.9, 1],

    'gamma': [0, 0.1, 0.2, 0.5],

    'reg_alpha': [0, 0.001, 0.01, 0.1, 1],

    'reg_lambda': [0.1, 1, 5, 10],

}

random_search =

    RandomizedSearchCV( estimator=model,

    param_distributions=param_dist,

    n_iter=50,          # Number of different combinations to try

    scoring='accuracy', # Metric to optimize

    cv=3,              # 3-fold cross-validation

    verbose=2,

    random_state=42,

    n_jobs=-1          # Use all cores

)

random_search.fit(x_train, y_train)

```

```

# grid_search = GridSearchCV(
#     estimator=model,
#     param_grid=param_dist,
#     scoring='accuracy',
#     cv=3,          # 3-fold cross-validation
#     verbose=2,
#     n_jobs=-1      # Use all available cores
# )

# grid_search.fit(x_train, y_train)

print("Best parameters found: ", random_search.best_params_)

print("Best cross-validation score: {:.4f}".format(random_search.best_score_))

# Train AdaBoost Classifier

from sklearn.ensemble import AdaBoostClassifier

from sklearn.model_selection import cross_val_score

abc = AdaBoostClassifier()

np.mean(cross_val_score(abc,X,Y,scoring='accuracy',cv=10)) abc
= AdaBoostClassifier(n_estimators=1800,learning_rate=0.2)
abc.fit(x_train,y_train)

# plot_decision_boundary(abc)

y_pred=abc.predict(x_test)

```

```

print("Accuracy score",accuracy_score(y_test,y_pred))

cm = confusion_matrix(y_test, y_pred, labels=[1, 0])

TP, FN, FP, TN = cm.ravel()

print("Modified Confusion Matrix (TP in [0][0]):")
print(f'[[TP: {TP}  FN: {FN}]]')
print(f' [FP: {FP}  TN: {TN}]]')
print("\nClassification Report:\n", classification_report(y_test, y_pred))

# Trasing Gradient Boosting Classifier

from sklearn.ensemble import GradientBoostingClassifier

gb_classifier = GradientBoostingClassifier(
    subsample=0.9,
    n_estimators=100,
    min_samples_split=5,
    min_samples_leaf=2,
    max_depth=3,
    learning_rate=0.05,
    random_state=42
)

# Train the model

gb_classifier.fit(x_train, y_train)

# Predict

```

```

y_pred = gb_classifier.predict(x_test)

# Evaluate

from sklearn.metrics import accuracy_score

print("Test Accuracy: ", accuracy_score(y_test, y_pred))

cm = confusion_matrix(y_test, y_pred, labels=[1, 0])

TP, FN, FP, TN = cm.ravel()

print("Modified Confusion Matrix (TP in [0][0]):")

print(f"[[TP: {TP}  FN: {FN}]]")

print(f"[[FP: {FP}  TN: {TN}]]")

print("\nClassification Report:\n", classification_report(y_test, y_pred))

# Train Stacking Classifier

from sklearn.ensemble import StackingClassifier

estimators = [

                                                                    ('rf',
RandomForestClassifier(max_depth=12,max_features=0.3,max_samples=1.0,n_estimators=1
00,bootstrap=True)),
    ('adb', AdaBoostClassifier(n_estimators=1500,learning_rate=0.1)),
    ('gdbt', GradientBoostingClassifier(subsample=0.9,n_estimators=100,min_samples_split=5,
min_samples_leaf=2,max_depth=3,learning_rate=0.05,random_state=42))

]

clf =

    StackingClassifier( estimato

rs=estimators,

```

```
final_estimator=xgb.XGBClassifier(subsample=0.7,reg_lambda=0.1,reg_alpha=0.001,n_estimators=100,max_depth=6,learning_rate=0.01,gamma=0,colsample_bytree=0.8,eval_metric='logloss'),
```

```
cv=10
```

```
)
```

```
clf.fit(x_train,y_train)
```

```
y_pred=clf.predict(x_test)
```

```
print('Accuracy SCore',accuracy_score(y_test,y_pred))
```

```
cm = confusion_matrix(y_test, y_pred, labels=[1, 0])
```

```
TP, FN, FP, TN = cm.ravel()
```

```
print("Modified Confusion Matrix (TP in [0][0]):")
```

```
print(f'[[TP: {TP} FN: {FN}]]')
```

```
print(f' [FP: {FP} TN: {TN}]]')
```

```
plt.show()
```

```
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

```
# Balanced RandomForest
```

```
from imblearn.ensemble import BalancedRandomForestClassifier
```

```
# Corrected classifier instantiation
```

```
brf =
```

```
BalancedRandomForestClassifier( sa
```

```
mpling_strategy=0.8,
```

```
n_estimators=300,
```

```
min_samples_split=2,
```

```

min_samples_leaf=6,

max_features='log2',

max_depth=50,

random_state=42 # Always good to set for reproducibility
)

brf.fit(x_train, y_train)

y_pred = brf.predict(x_test)

# Accuracy

print("Accuracy:", accuracy_score(y_test, y_pred))

cm = confusion_matrix(y_test, y_pred, labels=[1, 0])

TP, FN, FP, TN = cm.ravel()

print("Modified Confusion Matrix (TP in [0][0]):")

print(f"[[TP: {TP}  FN: {FN}]]")

print(f" [FP: {FP}  TN: {TN}]]")

plt.show()

print("\nClassification Report:\n", classification_report(y_test, y_pred))

```

Chapter 8: SIMILARITY REPORT